

KI-DUELL

Ressourcenstrategie & Seekrieg — Konzeptdokument v2.0

Zero-Player-Game im Browser | Zwei autonome KI-Spieler | Lokaler Stack (Ollama + qwen3-tts)

1. Architektur & Technologie-Stack

Das System ist als reines lokales Zero-Player-Game im Browser konzipiert. Der Benutzer agiert ausschließlich als Spielleiter und Zuschauer.

Schicht	Rolle & Technologie
Frontend	HTML / CSS / JavaScript: Game-Loop, Spiellogik (Schiffe-versenken-Raster), Wirtschaftsmathematik, SoundManager, UI-Rendering
PHP-Middleware	Lokaler Proxy-Server: nimmt fetch()-Anfragen von JS entgegen und leitet sie an Ollama und qwen3-tts weiter (umgeht CORS-Beschränkungen)
KI-Text	Ollama (lokal): JSON-Generierung für Spielentscheidungen + Dialogtext. Optional externe Anbieter via API-Key (Anthropic, DeepSeek, OpenAI)
KI-Audio	qwen3-tts (lokal, venv unter mnt/Daten/KI/qwen3-tts): Sprachsynthese aus dem Dialogtext jedes Zuges
Datenspeicher	localStorage des Browsers: API-Keys, Prompts, Persönlichkeitswerte, kompletter Spielstand als JSON

► Externe API-Keys sind rein optional und dienen dem Testen alternativer Modelle. Der Regelbetrieb funktioniert vollstaendig offline.

2. Benutzeroberfläche (UI)

Der Bildschirm ist in drei vertikale Bereiche aufgeteilt:

- Linke Hälfte (Spieler 1): Wirtschaftsdaten (Korn, Gold, Bevölkerung), eigenes Battleship-Raster (10x10) mit eigenen Schiffen und Treffern/Fehlschüssen des Gegners.
- Rechte Hälfte (Spieler 2): Identischer Aufbau für den zweiten KI-Spieler.

- Zentrale Konsole (Spielleiter): Start/Pause/Reset-Schaltflächen, Eingabefelder für optionale API-Keys, Schieberegler für KI-Persönlichkeiten (veränderbar nur vor oder nach einem Reset, nicht mid-game). Darunter läuft ein Text-Log mit Dialogen und KI-Entscheidungen (ausklappbare Rohdaten-Ansicht für Debugging).

🚩 *Das Battleship-Raster hat fest definierte Abmessungen von 10x10 Feldern (Spalten A-J, Zeilen 1-10).*

3. KI-Persönlichkeiten & Steuerung

Jede KI besitzt zwei unabhängig einstellbare Parameter (Skala 0-100), die getrennt in den System-Prompt eingespeist werden:

Parameter	Wirkungsbereich
Strategische Aggressivität (0-100)	Beeinflusst harte Spielentscheidungen: Steuersatz, Korn-Gold-Verhältnis, Priorisierung des Flottenaufbaus. Hoher Wert = hohe Steuern, Hunger der Bevölkerung, maximale Rüstungsgeschwindigkeit.
Verbale Aggressivität / Derbheit (0-100)	Beeinflusst ausschliesslich den Dialogtext. Niedriger Wert = diplomatisch, lobend. Hoher Wert = Beleidigungen, arroganter Trash-Talk. Kein Einfluss auf Spielentscheidungen.

Dadurch sind orthogonale Kombinationen möglich, z.B. ein defensiv spielender Strategie, der den Gegner permanent wüst beschimpft.

Persönlichkeitswerte sind nur vor Spielstart oder nach einem Reset änderbar, um konsistente Spielverläufe zu gewährleisten.

4. Spielmechanik & Phasen

Das Spiel verläuft rundenbasiert in "Jahren". Pro Runde durchläuft jeder Spieler drei Phasen sequenziell.

Phase 1: Wirtschaft

Die KI erhält einen Prompt mit dem Rundenstatus (eigenes Korn, Gold, Bevölkerung, sichtbarer Gegner-Status, Persönlichkeitswerte) und gibt ein valides JSON-Objekt zurück mit folgenden Feldern:

JSON-Feld	Bedeutung & Konsequenz
-----------	------------------------

steuern	Abgaben der Bevölkerung. Generiert Gold; bei zu hohen Werten sinkt Bevölkerungszufriedenheit und -zahl.
aussaat	Kornmenge aufs Feld. Bestimmt die Ernte der Folgerunde (Multiplikator: Erntequotient, s. Balancing-Konstanten).
ernaehrung	Korn an Bevölkerung. Unterschreitung des Mindestbedarfs führt zu Bevölkerungsschwund.
handel	Korn direkt zu Gold umgewandelt (Umrechnungskurs: Balancing-Konstante).
dialog	Dialogtext fuer qwen3-tts gemaess verbaler Aggressivitaet (s. Phase 3).
angriff	Zielkoordinate (z.B. "C4") falls Flotte vorhanden, sonst null.

Balancing-Konstanten (separates JS-Konfigurationsobjekt)

Alle Spielwerte sind in einem zentralen GAME_CONFIG-Objekt gebündelt, damit sie ohne Code-Suche anpassbar sind:

- Startwerte: KORN_START, GOLD_START, BEVOELKERUNG_START
- Erntequotient: ERNTE_FAKTOR (z.B. 2.5 = 2,5 Korn Ernte pro ausgesätem Korn)
- Handelskurs: HANDEL_KURS (Korn zu Gold, z.B. 0.5)
- Bevölkerungsbedarf: ERNAEHRUNG_PRO_KOPF (Korn/Runde pro Person)
- Flottenkosten: FLOTTE_GOLD_KOSTEN, FLOTTE_UNTERHALT_KORN (pro Runde)

🚩 *Fehlende oder invalide JSON-Felder werden mit Fallback-Werten (0 bzw. null) aufgefüllt, nie als Fehler behandelt.*

Phase 2: Rüstung & Seekrieg

Das Battleship-Raster hat 10x10 Felder. Schiffe werden beim Flottenkauf zufällig platziert, für den Gegner unsichtbar.

- Flottenkauf: Sobald ein Spieler die Schwelle FLOTTE_GOLD_KOSTEN erreicht, wird einmalig pro Spiel eine Flotte gekauft. Schiffe werden automatisch platziert. Ab diesem Zeitpunkt wird jede Runde FLOTTE_UNTERHALT_KORN abgezogen.
- Angriff: Hat ein Spieler eine Flotte, gibt die KI im JSON-Zug das Feld "angriff" mit einer Koordinate an. Das System prüft: Treffer, Fehlschuss oder versenktes Schiff.
- KI-Kontext für Battleship: Die KI kennt ihre eigene Treffer-/Fehlschuss-Karte aus allen Vorjahren (hitMap wird vollständig in den Prompt eingespeist). Ohne diesen Kontext würde die KI blind und zufällig schießen.

🚩 *hitMap im Prompt: Zweidimensionales Array oder kompakte Darstellung (z.B. "Treffer: B3, F7 | Fehlschuss: A1, C4"). Die KI muss bereits beschossene Felder explizit meiden.*

Phase 3: Kommunikation & Audio

Der Dialogtext aus Phase 1 wird asynchron an qwen3-tts gesendet. Die Spiellogik und UI-Aktualisierung laufen nicht auf die Audiogenerierung und warten nur auf das Abspielen der fertigen Datei. Ein "Audio überspringen"-Button erlaubt das Übergehen beim Testen oder bei langen TTS-Wartezeiten.

- TTS-Anfrage: PHP sendet den Dialogtext an qwen3-tts, erhält eine Audiodatei zurück.
- Abspielen: JavaScript spielt die Datei über das Audio-Objekt ab. Das "onended"-Ereignis startet den nächsten Spielerzug.
- Skip-Button: Unterbricht das Audio sofort und startet den nächsten Zug, ohne den TTS-Prozess abubrechen.

5. Siegbedingungen & Spielende

Ein Spieler gewinnt, wenn einer der folgenden Zustände eintritt. JavaScript prüft diese Bedingungen am Ende jeder Runde, bevor der nächste Zug beginnt.

Siegbedingung	Auslöser
Militärsieg	Alle Schiffe des Gegners wurden versenkt. Setzt voraus, dass der Gegner überhaupt eine Flotte besitzt; ist das nie der Fall, entfällt diese Siegbedingung und der Wirtschaftskollaps entscheidet.
Wirtschaftlicher Kollaps - Bevölkerung	Die Bevölkerung des Gegners erreicht 0. Das Spiel endet sofort in dieser Runde.
Wirtschaftlicher Kollaps - Gold	Das Gold des Gegners ist zwei Runden in direkter Folge negativ (nicht nur am Rundenende, sondern nach Abzug aller Kosten). Einmalig negatives Gold gilt als Warnung, nicht als Niederlage.

Nach Spielende wird eine Abschlussanzeige eingeblendet mit: Sieger, Siegbedingung, Rundenzahl, wirtschaftliche Endbilanz beider Spieler. Der Reset-Button setzt alle Variablen auf Startwerte zurück.

🚩 *Sonderfall Gleichzeitigkeit: Erfüllen beide Spieler in derselben Runde eine Siegbedingung (z.B. gegenseitiger Wirtschaftskollaps), endet das Spiel als Unentschieden.*

6. Sound-Modul & Fallback-Logik

Ein zentraler SoundManager in JavaScript verwaltet alle akustischen Ereignisse. Vor dem Abspielen wird per `fetch()` geprüft, ob die Datei existiert. Fehlt sie, greift die Web Audio API mit synthetischen Ersatzklängen.

Dateiname	Ereignis	Fallback (Web Audio API)
<i>ui_turn_start.mp3</i>	Rundenbeginn	Kurzer Sinus-Ping (880 Hz, 0,15 s)
<i>econ_coins.mp3</i>	Steuereinnahmen / Handel	Metallischer Glockenton (1.200 Hz, mit Decay)
<i>econ_harvest.mp3</i>	Ernte / Aussaat	Weiches Rauschen gefiltert (Bandpass 600 Hz)
<i>econ_famine.mp3</i>	Bevölkerungsschwund	Tiefer Moll-Akkord (220 Hz + 261 Hz, langsam)
<i>fleet_buy.mp3</i>	Flottenkauf	Tiefer Schiffshorn-Ton (110 Hz, langer Decay)
<i>combat_cannon.mp3</i>	Schuss abgefeuert	Tiefer Impulston mit Rauschen (80 Hz Burst)
<i>combat_splash.mp3</i>	Fehlschuss / Wasser	Weisses Rauschen, kurz (0,3 s), gefiltert
<i>combat_hit.mp3</i>	Treffer auf Schiff	Scharfer Impact-Ton (400 Hz + Oberton 800 Hz)
<i>combat_sink.mp3</i>	Schiff versenkt	Zweistufiger Ton: Impact + tiefer Abgang (80 Hz)

Alle Sounds liegen unter `/assets/sounds/` mit exakt diesen Dateinamen. Der Fallback ist vollstaendig in JavaScript implementiert und benoetigt keine externen Bibliotheken.

7. Technischer Game-Loop

Ablauf eines vollstaendigen Zuges (ein Spieler, ein Jahr):

- JavaScript sammelt den vollstaendigen Spielzustand beider Spieler: Ressourcen, hitMap (eigene Treffer/Fehlschüsse), Persönlichkeitswerte, Rundenhistorie fuer Goldstand-Prüfung.
- Prompt-Builder erstellt den Systemkontext (Persönlichkeit, Spielregeln) und den User-Prompt (Statusdaten) als separates JSON-Objekt.
- `fetch()` sendet den Prompt an die PHP-Middleware.
- PHP leitet den Request an Ollama weiter und gibt die Antwort als Stream oder komplett zurück.
- JavaScript parst die KI-Antwort. Bei ungültigem JSON: bis zu 3 Retry-Versuche mit explizitem Fehler-Hinweis im Prompt. Nach 3 Fehlversuchen: Fallback-Zug (neutrale Ressourcenverteilung, kein Angriff).
- Spiellogik berechnet Rundeneffekte: Ernte, Steuerertrag, Bevölkerungsentwicklung, Flottenunterhalt-Abzug, Battleship-Auswertung.

7. SoundManager spielt die passenden Sounds sequenziell ab (Wirtschaft, dann Kampf).
8. UI wird aktualisiert: Ressourcenzähler, Raster, Log (Klartext + ausklappbare JSON-Rohdaten).
9. Siegbedingungen werden geprüft. Bei Spielende: Abschlussanzeige einblenden und Loop stoppen.
10. TTS-Text wird asynchron per PHP an qwen3-tts gesendet. Audio wird abgespielt. Nach "onended" oder Skip-Klick: Nächster Spieler startet bei Schritt 1.

8. Fehlerbehandlung & Robustheit

Alle kritischen Stellen im Game-Loop sind mit Fehlerbehandlung abgesichert, damit der Spielfluss nicht abbricht:

- JSON-Parsing: try/catch um JSON.parse(). Bei Fehler: Retry-Prompt mit explizitem Hinweis ("Antwort war kein valides JSON, bitte nur das JSON-Objekt zurückgeben"). Nach 3 Fehlversuchen: Fallback-Zug, Fehlermeldung im Log.
- Fehlende JSON-Felder: Jedes Feld wird einzeln geprüft und mit Standardwert (0, null) aufgefüllt, nie als Absturzursache behandelt.
- Netzwerk-/Ollama-Fehler: fetch()-Fehler werden abgefangen. Im Log erscheint ein Hinweis, der Zug wird übersprungen.
- TTS-Fehler: Schlägt qwen3-tts fehl, wird der Dialog nur als Text im Log angezeigt. Der Spielfluss läuft ohne Audio weiter.
- Sound-Fallback: Existiert eine Sounddatei nicht, generiert die Web Audio API einen synthetischen Ersatz (s. Abschnitt 6). Der SoundManager fängt alle Audio-Fehler ab.
- Overflow-Schutz: Ressourcenwerte werden nach jeder Runde auf definierte Maximalwerte geclamped (GAME_CONFIG.MAX_KORN, MAX_GOLD) um Integer-Overflow in langen Spielen zu vermeiden.

9. Implementierungshinweise & empfohlene Reihenfolge

Empfohlene Entwicklungsreihenfolge um schnell zu einem lauffähigen Stand zu kommen:

11. GAME_CONFIG-Objekt definieren: Alle Balancing-Konstanten und Startwerte.
12. Wirtschaftslogik implementieren und mit festen Testwerten (kein LLM) durchsimulieren.
13. Battleship-Raster implementieren: Platzierung, hitMap, Trefferauswertung.
14. Siegbedingungen implementieren und mit künstlich herbeigeführten Endzuständen testen.
15. JSON-Prompt-System mit Retry-Logik und Fallback einbauen.
16. PHP-Middleware und Ollama-Anbindung hinzufügen.
17. SoundManager mit Fallback-Synthese, dann qwen3-tts-Anbindung.
18. UI-Feinschliff, Log mit Rohdaten-Toggle, Skip-Button.

🚩 *Die hitMap-Integration (Schritt 3/5) ist kritisch fuer sinnvolles KI-Verhalten im Seekrieg und sollte nicht als nachtraegliche Erweiterung behandelt werden.*

Dokument: KI-Duell Konzept v2.0 | Erstellt mit Claude Sonnet 4.6 | Mai 2025